

ACE-Step 1.5 LoRA Training Tutorial

Hardware Requirements

VRAM	Description
16 GB (minimum)	Generally sufficient, but longer songs may cause out-of-memory errors
20 GB or more (recommended)	Handles full-length songs; VRAM usage typically stays around 17 GB during training

> **Note:** During the preprocessing stage before training, you may need to restart Gradio multiple times to free VRAM. The specific timing will be mentioned in the steps below.

Disclaimer

This tutorial uses the album **ナユタン星からの物体Y** by **Nayutan 星人 (NayutalieN)** (13 tracks) as a demonstration, trained for 500 epochs (batch size 1). **This tutorial is intended solely for educational purposes to understand LoRA fine-tuning. Please use your own original works to train your LoRA.**

As a developer, I personally enjoy NayutalieN's work and chose one of their albums as an example. If you are the rights holder and believe this tutorial infringes upon your legitimate rights, please contact us immediately. We will remove the relevant content upon receiving a valid notice.

Technology should be used reasonably and lawfully. Please respect artists' creations and refrain from any actions that **harm or damage** the reputation, rights, or interests of original artists.

Data Preparation

> **Tip:** If you are unfamiliar with programming, you can provide this document to AI coding tools such as Claude Code / Codex CLI / Cursor / Copilot and let them handle the scripting tasks for you.

Overview

Training data for each song consists of the following:

- Audio file** — Supported formats: `.mp3`, `.wav`, `.flac`, `.ogg`, `.opus`
- Lyrics** — A `.lyrics.txt` file with the same name as the audio (`.txt` is also supported for backward compatibility)
- Annotation data** — Metadata including `caption`, `bpm`, `keyscale`, `timesignature`, `language`, etc.

Annotation Data Format

If you already have complete annotation data, you can create JSON files and place them in the same directory as the audio and lyrics. The file structure is as follows:

```

...
dataset/
├── song1.mp3          # Audio
├── song1.lyrics.txt   # Lyrics
├── song1.json         # Annotations (optional)
├── song1.caption.txt  # Caption (optional, can also be included in JSON)
├── song2.mp3
├── song2.lyrics.txt
├── song2.json
└── ...
...

```

JSON file structure (all fields are optional):

```

```json
{
 "caption": "A high-energy J-pop track with synthesizer leads and fast tempo",
 "bpm": 190,
 "keyscale": "D major",
 "timesignature": "4",
 "language": "ja"
}
```

```

If you don't have annotation data, you can obtain it using the methods described in later sections.

Lyrics

Save lyrics as a `.lyrics.txt` file with the same name as the audio file, placed in the same directory. Please ensure the lyrics are accurate.

Lyrics file lookup priority during scanning:

1. `{filename}.lyrics.txt` (recommended)
2. `{filename}.txt` (backward compatible)

Lyrics Transcription

If you don't have existing lyrics text, you can obtain transcribed lyrics using the following tools:

| Tool | Structural Tags | Accuracy | Ease of Use | Deployment |
|--|-----------------|--------------------|---|------------------------|
| [acestep-transcriber](https://huggingface.co/ACE-Step/acestep-transcriber) | No | May contain errors | High difficulty (requires model deployment) | Self-hosted |
| [Gemini](https://aistudio.google.com/) | Yes | May contain errors | Easy | Paid API |
| [Whisper](https://github.com/openai/whisper) | No | May contain errors | Moderate | Self-hosted / Paid API |

| [ElevenLabs](https://elevenlabs.io/app/developers) | No | May contain errors | Moderate | Paid API (generous free tier) |

This project provides transcription scripts under `scripts/lora\_data\_prepare/`:

- `whisper\_transcription.py` — Transcription via OpenAI Whisper API
- `elevenlabs\_transcription.py` — Transcription via ElevenLabs Scribe API

Both scripts support the `process\_folder()` method for batch processing entire folders.

Review and Cleanup (Required)

Transcribed lyrics may contain errors and **must be manually reviewed and corrected**.

If you are using LRC format lyrics, you need to remove the timestamps. Here is a simple cleanup example:

```
```python
import re

def clean_lrc_content(lines):
 """Clean LRC file content by removing timestamps"""
 result = []
 for line in lines:
 line = line.strip()
 if not line:
 continue
 # Remove timestamps [mm:ss.x] [mm:ss.xx] [mm:ss.xxx]
 cleaned = re.sub(r"[\d{2}:\d{2}\.\d{1,3}]", "", line)
 result.append(cleaned)

 # Remove trailing empty lines
 while result and not result[-1]:
 result.pop()

 return result
```
```

Structural Tags (Optional)

Including structural tags in lyrics (such as `[Verse]`, `[Chorus]`, etc.) helps the model learn song structure more effectively. Training without structural tags is also possible.

> **Tip:** You can use [Gemini](https://aistudio.google.com/) to add structural tags to existing lyrics.

Example:

```
```
[Intro]
La la la...
```
```

[Verse 1]

Walking down the empty street
Echoes dancing at my feet

[Chorus]

We are the stars tonight
Shining through the endless sky

[Bridge]

Close your eyes and feel the sound

Automatic Annotation

1. Obtaining BPM and Key

Use [Key-BPM-Finder](https://vocalremover.org/key-bpm-finder) to obtain BPM and key annotations online:

1. Open the webpage and click ****Browse my files**** to select the audio files to process (processing too many at once may cause the page to freeze — batch processing and merging CSVs is recommended). Processing is done locally and files are not uploaded to a server.

![key-bpm-finder-0.jpg](../pics/key-bpm-finder-0.jpg)

2. After processing, click ****Export CSV**** to download the CSV file.

![key-bpm-finder-1.jpg](../pics/key-bpm-finder-1.jpg)

3. CSV file content example:

```
```csv
File,Artist,Title,BPM,Key,Camelot
song1.wav,,,190,D major,10B
song2.wav,,,128,A minor,8A
```
```

4. Place the CSV file in the dataset folder. To include caption data, add an extra column after `Camelot`.

2. Obtaining Captions

Captions can be obtained in the following ways:

- ****Using acesep-5Hz-lm**** (0.6B / 1.7B / 4B) — Via the Auto Label feature in the Gradio UI (see subsequent steps)

- ****Using Gemini API**** — Refer to the script `scripts/lora\_data\_prepare/gemini\_caption.py`, which supports `process\_folder()` for batch processing and generates the following for each audio file:

- `{filename}.lyrics.txt` — Lyrics
- `{filename}.caption.txt` — Caption description

Data Preprocessing

Once data is prepared, you can use the Gradio UI for data review and preprocessing.

```
> **Important:** If using a startup script, you need to modify the launch parameters to
disable service pre-initialization:
>
> - **Windows** (`start_gradio_ui.bat`): Change `if not defined INIT_SERVICE set
INIT_SERVICE=--init_service true` to `if not defined INIT_SERVICE set
INIT_SERVICE=--init_service false`
> - **Linux/macOS** (`start_gradio_ui.sh`): Change `:"${INIT_SERVICE:=--init_service
true}"` to `:"${INIT_SERVICE:=--init_service false}"`
```

Launch the Gradio UI (via the startup script or by running `acestep/acestep\_v15\_pipeline.py` directly).

Step 1: Load Models

- ****If you need to use LM for caption generation:**** Select the desired LM model during initialization (acestep-5Hz-lm-0.6B / 1.7B / 4B).

- ****If you don't need LM:**** Do not select any LM model.

Step 2: Load Data

Switch to the ****LoRA Training**** tab, enter the dataset directory path, and click ****Scan****.

The scanner automatically recognizes the following files:

| File | Description |
|---|---|
| ----- ----- | |
| `*.mp3` / `*.wav` / `*.flac` / ... | Audio files |
| `{filename}.lyrics.txt` (or `{filename}.txt`) | Lyrics |
| `{filename}.caption.txt` | Caption description |
| `{filename}.json` | Annotation metadata (caption / bpm / keyscale / timesignature / language) |
| `*.csv` | Batch BPM / Key annotations (exported from Key-BPM-Finder) |

Step 3: Review and Adjust Dataset

- ****Duration**** — Automatically read from the audio file
- ****Lyrics**** — Requires a corresponding `.lyrics.txt` file (`.txt` is also supported)
- ****Labeled**** — Shows  if caption exists, ✕ otherwise
- ****BPM / Key / Caption**** — Loaded from JSON or CSV files
- If the dataset is not entirely instrumental, uncheck ****All Instrumental****

- **Format Lyrics** and **Transcribe Lyrics** are currently disabled (not yet integrated with [acestep-transcriber](https://huggingface.co/ACE-Step/acestep-transcriber); using LM directly tends to produce hallucinations)
- Enter a **Custom Trigger Tag** (currently has limited effect; any option other than `Replace Caption` is fine)
- **Genre Ratio** controls the proportion of samples using genre instead of caption. Since the current LM-generated genre descriptions are far less descriptive than captions, keep this at 0

Step 4: Auto Label Data

- If you already have captions, you can skip this step
- If your data lacks captions, use LM inference to generate them
- If BPM / Key values are missing, obtain them via [Key-BPM-Finder](https://vocalremover.org/key-bpm-finder) first — generating them directly with LM will produce hallucinations

Step 5: Review and Edit Data

If needed, you can review and modify data entry by entry. **Remember to click Save after editing each entry.**

Step 6: Save Dataset

Enter a save path to export the dataset as a JSON file.

Step 7: Preprocess and Generate Tensor Files

> **Note:** If you previously used LM to generate captions and VRAM is insufficient, restart Gradio to free VRAM first. When restarting, **do not select the LM model**. After restarting, enter the path to the saved JSON file and load it.

Enter the save path for tensor files, click to start preprocessing, and wait for it to complete.

Training

> **Note:** After generating tensor files, it is also recommended to restart Gradio to free VRAM.

1. Switch to the **Train LoRA** tab, enter the tensor file path, and load the dataset.

2. If you are unfamiliar with training parameters, the default values are generally fine.

Parameter Reference

| Parameter | Description | Suggested Value |
|-------------------------|--|--|
| ----- ----- ----- | | |
| **Max Epochs** | Adjust based on dataset size | ~100 songs → 500 epochs; 10–20 songs → 800 epochs (for reference only) |
| **Batch Size** | Can be increased if VRAM is sufficient | 1 (default); try 2 or 4 if VRAM allows |
| **Save Every N Epochs** | Checkpoint save interval | Set smaller for fewer Max Epochs, larger for more |

> The above values are for reference only. Please adjust based on your actual situation.

>  **LoKr Recommendation:** LoKR has greatly improved training efficiency. What used to take an hour now only takes 5 minutes—over 10 times faster. This is crucial for training on consumer-grade GPUs. You can try LoKr training in the **Train LoKr** tab, or use the [Side-Step](https://github.com/koda-dernet/Side-Step) toolkit for CLI-based LoKr workflows. See the [Training Guide](../sidestep/Training%20Guide.md) for details.

3. Click **Start Training** and wait for training to complete.

LoKr Training (Alternative to LoRA)

LoKr (Low-rank Kronecker product) is an alternative adapter method that uses Kronecker decomposition instead of low-rank matrix factorization. Compared to standard LoRA, LoKr offers significantly faster training times -- what previously took an hour can often complete in around 5 minutes -- making it particularly well-suited for consumer-grade GPUs.

How LoKr Differs from LoRA

| Aspect | LoRA | LoKr |
|-----------------------|-------------------------------|---|
| ----- ----- ----- | | |
| Decomposition | Low-rank matrix factorization | Kronecker product decomposition |
| Training speed | Baseline | Up to 10x faster |
| Default learning rate | 1e-4 | 0.03 |
| Default epochs | 10 | 500 |
| DoRA support | No | Yes (enabled by default via `weight_decompose`) |

LoKr Parameters

Adapter-Specific Parameters

| Parameter | Default | Description |
|-----------------------|---------|---|
| ----- ----- ----- | | |
| **lokr_linear_dim** | 64 | Linear dimension of the LoKr adapter (1-256) |
| **lokr_linear_alpha** | 128 | Linear alpha scaling factor (1-512, typically 2x dim) |

| ****lokr_factor**** | -1 (auto) | Kronecker factor. -1 lets the algorithm choose automatically.
Valid range: 1-8 when set manually |
| ****lokr_decompose_both**** | false | Decompose both matrices in the Kronecker product |
| ****lokr_use_tucker**** | false | Use Tucker decomposition for additional compression |
| ****lokr_use_scalar**** | false | Enable scalar calibration |
| ****lokr_weight_decompose**** | true | Enable DoRA (Weight-Decomposed Low-Rank Adaptation) mode |

Shared Training Parameters

These parameters are shared between LoRA and LoKr training:

| Parameter | LoRA Default | LoKr Default | Description |
|-----------------------------------|--------------|--------------|--|
| **learning_rate** | 1e-4 | 0.03 | Optimization learning rate |
| **train_epochs** | 10 | 500 | Maximum training epochs |
| **train_batch_size** | 1 | 1 | Training batch size |
| **gradient_accumulation** | 4 | 4 | Gradient accumulation steps (effective batch = batch_size x accumulation) |
| **save_every_n_epochs** | 5 | 5 | Checkpoint save frequency |
| **training_shift** | 3.0 | 3.0 | Timestep shift for turbo model |
| **training_seed** | 42 | 42 | Random seed for reproducibility |
| **gradient_checkpointing** | false | false | Trade compute speed for lower VRAM usage. Enable this if you are running out of VRAM during training |
| **use_fp8** | false | N/A | Use FP8 precision when the runtime supports it (LoRA only) |

Using LoKr in the Gradio UI

1. Switch to the ****Train LoKr**** tab (next to the Train LoRA tab)
2. Load your preprocessed tensor dataset (same tensors used for LoRA training)
3. Adjust LoKr-specific parameters if needed (defaults work well for most cases)
4. Click ****Start Training**** and monitor the loss curve
5. Export the trained adapter when training completes

Using LoKr via the API

LoKr training is also available through the HTTP API:

```
```bash
curl -X POST http://localhost:8001/v1/training/start_lokr \
 -H 'Content-Type: application/json' \
 -d '{
 "tensor_dir": "/path/to/preprocessed/tensors",
 "output_dir": "./lokr_output",
 "lokr_linear_dim": 64,
 "lokr_linear_alpha": 128,
 "lokr_factor": -1,
 "lokr_weight_decompose": true,
 "learning_rate": 0.03,
 "train_epochs": 500,
 "train_batch_size": 1,
 "gradient_accumulation": 4,
```



```

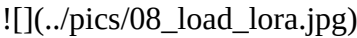
 "save_every_n_epochs": 5
 }'
'''

```

The endpoint mirrors the LoRA training API (`POST /v1/training/start``), with LoKR-specific parameters replacing the LoRA rank/alpha/dropout fields. Training progress can be monitored using the same status and stop endpoints used for LoRA training.

---

## ## Using LoRA

1. After training completes, **restart Gradio** and reload models (do not select the LM model).
2. Once the model is initialized, load the trained LoRA weights.  

3. Start generating music.

Congratulations! You have completed the entire LoRA training workflow.

---

## ## Advanced Training with Side-Step

For users who want more control over LoRA training — including corrected timestep sampling, LoKR adapters, CLI-based workflows, VRAM optimization, and gradient sensitivity analysis — the community-developed **[Side-Step]**(<https://github.com/koda-dernet/Side-Step>) toolkit provides an advanced alternative. Its documentation is bundled in this repository under ``docs/sidestep/``.

Topic	Description
[Getting Started](../sidestep/Getting%20Started.md)	Installation, prerequisites, and first-run setup
[End-to-End Tutorial](../sidestep/End-to-End%20Tutorial.md)	Complete walkthrough from raw audio to generation
[Dataset Preparation](../sidestep/Dataset%20Preparation.md)	JSON schema, audio formats, metadata fields, custom tags
[Training Guide](../sidestep/Training%20Guide.md)	LoRA vs LoKR, corrected vs vanilla mode, hyperparameter guide
[Using Your Adapter](../sidestep/Using%20Your%20Adapter.md)	Output layout, loading in Gradio, LoKR limitations
[VRAM Optimization Guide](../sidestep/VRAM%20Optimization%20Guide.md)	GPU memory profiles and optimization strategies
[Estimation Guide](../sidestep/Estimation%20Guide.md)	Gradient sensitivity analysis for targeted training
[Shift and Timestep Sampling](../sidestep/Shift%20and%20Timestep%20Sampling.md)	How training timesteps work and why Side-Step differs from the built-in trainer
[Preset Management](../sidestep/Preset%20Management.md)	Built-in presets, save/load/import/export
[The Settings Wizard](../sidestep/The%20Settings%20Wizard.md)	Complete wizard settings reference

| [Model Management](../sidestep/Model%20Management.md) | Checkpoint structure and fine-tune support |  
| [Windows Notes](../sidestep/Windows%20Notes.md) | Windows-specific setup and workarounds |